Assignment 5

1. The graph shows a **nonlinear increasing curve** that becomes steeper as increases. This is characteristic of an **exponential model**.
   The appropriate model is:

$$\boxed{y = x^w \text{ with } w > 0}$$

But note: among the given options, the notation in the image appears to use x^w.
Since the curve is **convex upward** and rapidly increasing, the intended choice is

$$y = x^w \text{ with } w > 0.$$

Answer: 3

2. For a neural network with:
   5 input variables
   1 hidden layer with 3 neurons
   1 output neuron for binary classification
   Bias terms counted as parameters
   Step 1: Input → Hidden layer
   Each of the 3 hidden neurons receives 5 inputs: 5*3 = 15
   Each hidden neuron has one bias:
   So: Total 3 bias. 15+3 = 18
   Step 2: Hidden → Output layer
   The output neuron receives 3 hidden-neuron outputs: 3 weights
   Plus one output bias: So total 3+1= 4
   Total parameters 18+4 =22
3. For a normal distribution with unknown mean μ, the **maximum likelihood estimator (MLE) of the mean is the sample mean**:

$$\hat{\mu}_{MLE} = \bar{x}$$

Given marks: 25,55,64, 7,99.

Calculate the mean:

$$\bar{x} = \frac{25 + 55 + 64 + 7 + 99}{5}$$

$$= \frac{250}{5} = 50$$

Therefore, the answer is 50

4. The weighted sum at the neuron is

$$z = (-4)(1) + 8x_1 + 8x_2$$

The threshold activation is

$$h(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Let's check all possible inputs:

x_1	x_2	$z = -4 + 8x_1 + 8x_2$	Output
0	0	-4	0
0	1	4	1
1	0	4	1
1	1	12	1

Thus, the output is **1 whenever at least one input is 1**.

$$\boxed{y = x_1 \vee x_2}$$

So the answer is OR

5. For the **3–3–1 architecture**, we have: 2 input variables + 1 bias node, 2 hidden neurons + 1 bias node, Sigmoid activation at both layers.
Given:

$$\alpha = \begin{bmatrix} 1 & 0.2 & 0.4 \\ -1 & 0.8 & 0.5 \end{bmatrix}$$

And

$$\beta = [0.8 \quad 0.4 \quad 0.5]$$

Input:

$$x = (0.8, 0.7)$$

Step 1: Hidden layer

Augmented input:

$$X = [1,\ 0.8,\ 0.7]^T$$

For hidden neuron 1:

$$z_1 = 1 + (0.2)(0.8) + (0.4)(0.7) = 1.44$$

$$h_1 = \sigma(1.44) = \frac{1}{1 + e^{-1.44}} \approx 0.8085$$

For hidden neuron 2:

$$z_2 = -1 + (0.8)(0.8) + (0.5)(0.7) = -0.01$$

$$h_2 = \sigma(-0.01) \approx 0.4975$$

Step 2: Output layer

Including the bias:

$$z_o = 0.8 + (0.4)(0.8085) + (0.5)(0.4975)$$

$$z_o \approx 1.3725$$

Apply sigmoid:

$$y = \sigma(1.3725) = \frac{1}{1 + e^{-1.3725}} \approx \boxed{0.7977}$$

So the answer is 0.7977.

6. The first statemen states that, The chances of overfitting decreases with increasing the number of hidden nodes and increasing the number of hidden layers. Increasing hidden nodes/layers increases the model's complexity and capacity, which can **increase the chance of overfitting**, especially with limited training data. So it is false.
   Statement 2 states that, A neural network with one hidden layer can represent any Boolean function given sufficient number of hidden units and appropriate activation functions. A single hidden-layer neural network can represent **any Boolean function** if enough hidden units and suitable activation functions are used. So it is True.

Statement 3 states that, Two hidden layer neural networks can represent any continuous functions (within a tolerance) as long as the number of hidden units is sufficient and appropriate activation functions are used.
This follows from the **Universal Approximation Theorem**. Neural networks with sufficient hidden units can approximate continuous functions to an arbitrary desired accuracy. So it is also True.

7. Given,

$$f(x, w) = \sigma\left(\sigma(x_1 w_1) w_2 + x_2\right)$$

Where,

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

We have:

$$w_1 = w_2 = 1, \qquad x_1 = 0, \qquad x_2 = 1$$

Step 1: Calculate the inner sigmoid,

$$\sigma(x_1 w_1) = \sigma(0 \times 1) = \sigma(0) = 0.5$$

Therefore, the input to the outer sigmoid is,

$$z = (0.5)(1) + 1 = 1.5$$

So,

$$\sigma(1.5) \approx 0.8176$$

Step 2: Differentiate with respect to

Using

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

we get

$$\frac{\partial f}{\partial w_2} = \sigma(1.5)[1 - \sigma(1.5)] \times \sigma(x_1 w_1)$$

$$= 0.8176(1 - 0.8176)(0.5)$$

$$\approx 0.0746$$

Therefore, the answer is 0.0746.

8. From the previous question we get,

$$\frac{\partial f}{\partial w_2} = 0.0746$$

Given:

$$w_2 = 1, \qquad \eta = 0.5, \qquad y = 5$$

The network output was,

$$f \approx 0.8176$$

The network output was ,

$$f \approx 0.8176$$

Using the squared-error loss,

$$E = (y - f)^2$$

the gradient is,

$$\frac{\partial E}{\partial w_2} = 2(f - y)\frac{\partial f}{\partial w_2}$$

$$= 2(0.8176 - 5)(0.0746)$$

$$\approx -0.624$$

Now apply the backpropagation update:

$$w_2^{new} = w_2 - \eta \frac{\partial E}{\partial w_2}$$

$$= 1 - (0.5)(-0.624)$$

$$\approx 1.312$$

So the answer is, 1.3119

9. The 1st statement is, ANN error surface has multiple local minima while SVM error surface has only one minimum.
   - ANN training is generally **non-convex**, so it may have multiple local minima/saddle points.
   - SVM optimization is a **convex optimization problem**, so it has a global optimum.

   So it is True.

The 2nd statement is After training, an ANN might land on a different minimum each time when initialized with random weights.
Different random initializations can lead gradient descent to different solutions/minima.
So it is also True.
Statement 3 is saying, Some classes of functions cannot be learned by an ANN. An SVM can learn a hyperplane for any kind of distribution.
The limitation described applies to a **single-layer perceptron**, not neural networks in general. Also, a standard SVM learns a linear hyperplane; kernels can allow nonlinear decision boundaries, but not literally "any kind of distribution." It is False.

The 4th Statement is, ANN uses gradient descent, while SVM uses convex optimization solvers.
ANNs are commonly trained using gradient-based optimization such as backpropagation + gradient descent, whereas SVM training is formulated as a convex optimization problem. So it is True.

10. 1 and 3 is correct.